

Reviewer Pack — Minimal Rank of Tropicalized Deligne-Lusztig Polynomials over...

Entry e634e3c466f3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 15:29:16 UTC

Minimal Rank of Tropicalized Deligne-Lusztig Polynomials over XOR-AND Tree Width

Entry ID: e634e3c466f3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 15:29:16 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Deligne-Lusztig Theory) **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

[‘For any XOR-AND tree with n leaves, the minimal rank of its associated tropicalized Deligne-Lusztig polynomial is at most $\Theta(\log(n))$.’, ‘The minimal rank of the tropicalized Deligne-Lusztig polynomial for an XOR-AND tree with n leaves is exactly $O(1)$ when $n \leq 40$.’, ‘For a fixed number k , there exists an XOR-AND tree with $n = 2^k + 3$ leaves such that the minimal rank of its tropicalized Deligne-Lusztig polynomial is $\Omega(n)$.’]

Rationale (proposer’s reasoning):

[‘Tropicalization provides a way to study algebraic structures in a geometric setting, which may reveal hidden complexities in XOR-AND trees.’, ‘The Deligne-Lusztig theory has connections to the representation theory of finite groups, and its tropicalized version might expose new properties relevant to computational complexity.’, ‘This conjecture aims to bridge the gap between the algebraic nature of Deligne-Lusztig polynomials and the geometric nature of XOR-AND trees, potentially leading to new insights into communication complexity.’]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bd04c8f4399cdfed

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

To support or falsify the conjecture, we will measure the minimal rank of tropicalized Deligne-Lusztig polynomials for XOR-AND trees with n leaves. A result supports the conjecture if the minimal rank is at most $\Theta(\log(n))$ for all $n \leq 40$ and $O(1)$ for $n = 2^k + 3$, $k \leq 5$. Falsification occurs if any tree with n leaves has a minimal rank exceeding these bounds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND Deligne-Lusztig theory AND XOR-AND tree width
- Deligne-Lusztig polynomials AND tropicalization AND complexity theory XOR-AND tree -
minimal rank AND tropicalized Deligne-Lusztig polynomials AND $O(1)$ XOR-AND tree

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_xor_and_tree(n):
        if n == 1:
            return 'x'
        else:
            left = generate_xor_and_tree(n // 2)
            right = generate_xor_and_tree(n - n // 2)
            return f'({left} {random.choice("&", "|")} {right})'

    def tropicalize(tree):
        if isinstance(tree, str):
            return tree
        else:
            left = tropicalize(tree[0])
            op = tree[1]
            right = tropicalize(tree[2])
            if op == '&':
                return f'({left} & {right})'
            elif op == '|':
                return f'({left} | {right})'

    def minimal_rank(tree):
        if isinstance(tree, str):
            return 0
        else:
            left_rank = minimal_rank(tree[0])
```

```

        right_rank = minimal_rank(tree[2])
        return max(left_rank, right_rank) + 1

n = random.randint(1, 40)
tree = generate_xor_and_tree(n)
rank = minimal_rank(tropicalize(eval(tree)))

return {
    "metric_name": "minimal_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": rank <= n.bit_length(),
    "counterexample": "" if rank <= n.bit_length() else f"rank={rank}, expected={n.bit_length()}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(6)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"rank exceeds expected\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d49b2e11.py", line 67, in <module>
    trial_result = run_trial(seed)
    ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d49b2e11.py", line 51, in run_trial
    rank = minimal_rank(tropicalize(eval(tree)))
    ^^^^^^^^^
  File "<string>", line 1, in <module>
NameError: name 'x' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which prevents us from verifying whether the conjecture’s conditions are met. | next: Re-run the test and ensure it completes successfully to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12269
2	preregistration	ollama_remote	glm4:latest	0	0	6591
3	novelty	ollama_remote	glm4:latest	0	0	4980
4	novelty	ollama_remote	glm4:latest	0	0	5591
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13290
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7590
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7379
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7034
9	judge	ollama_remote	glm4:latest	0	0	8375

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 73099 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/e634e3c466f3.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e634e3c466f3.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e634e3c466f3.tar.gz (if generated)